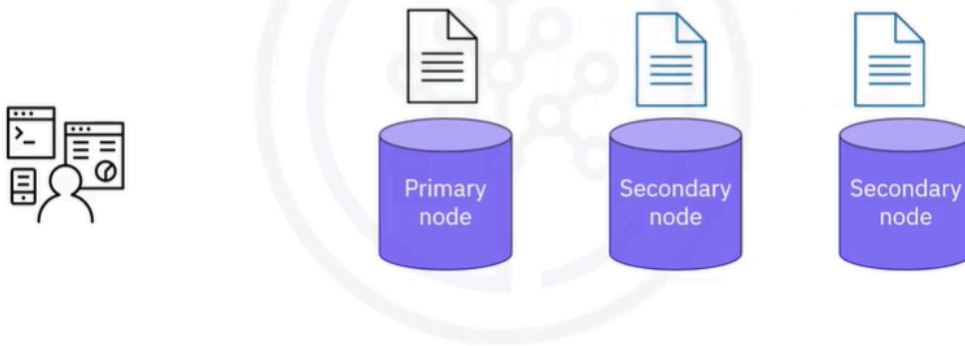


Replication and Sharding

Replication

A MongoDB Replica Set is a cluster of data-bearing nodes, each maintaining a copy of the same data

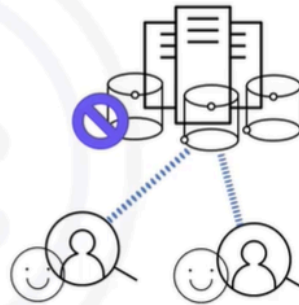


- A typical MongoDB cluster is made of three data-bearing nodes. All three nodes have the same data, hence the name Replica Set. Data is written to the primary node which then gets replicated to the secondary nodes.

Benefits of replication

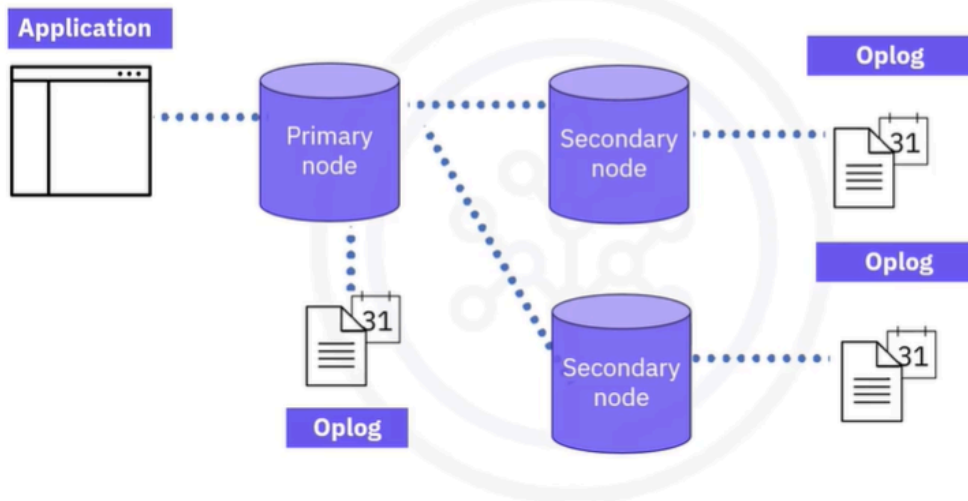
- Redundancy
- High availability
- Fault tolerance

Caution! Be aware that replication does not equal disaster recovery



- Having multiple copies of the data means that replication creates redundancy. So if one piece of server hardware fails, you still have multiple copies of the data. This capability provides you with a highly available database during such failures or plan maintenance periods. Such planned maintenance should be done in a rolling fashion, taking one node out for operating system, security, hardware, or software updates, or for upgrading MongoDB itself.
- A common misconception about replication is that replication can save you from disasters, such as accidentally deleting your database. Because the database is a replica set, what happens on the primary node is replicated on the secondary nodes. For disaster recovery scenarios, you'll rely on backups and restoration processes.

The replication process



- Here's an overview of the MongoDB replication process. The application writes changes to the primary node. The primary node records those changes including timestamps in its operations log known as the Oplog.
- All of the other data-bearing nodes, the secondary nodes, observe the primary node Oplog for new changes. The secondary nodes copy and apply the new changes by first recording those changes in their respective Oplogs with the time stamp that indicates when they receive the change and reference the primary Oplog timestamps to record only the most recent changes.

How an Oplog works

Example: A database of student information:

```
{ "_id": 1, "firstName": "Sophie", "lastName": "Foucher", "city": "Quebo" }  
{ "_id": 2, "firstName": "Stafford", "lastName": "von Hagt", "city": "Terezin" }  
{ "_id": 3, "firstName": "Olivero", "lastName": "Liggett", "city": "Guaranda" }
```

You can delete the preceding data using the statement:

```
db.students.deleteMany({})
```

- Let's examine a database that includes student information. You can see the three student IDs in the database. Next, you'll delete students from the database using the command `db.students.deleteMany`.

How an Oplog works

The command, `db.students.deleteMany({})` displays as an individual operation in the Oplog

```
{ "op" : "d", "ns" : "campusManagement.student", "o" : { "_id" : 1 } }  
{ "op" : "d", "ns" : "campusManagement.student", "o" : { "_id" : 2 } }  
{ "op" : "d", "ns" : "campusManagement.student", "o" : { "_id" : 3 } }
```

- Although you removed three students from the database, you will see the statement `db.students.deleteMany` as one individual operation in the Oplog.

What is Sharding?

When data is growing beyond hardware capacity:

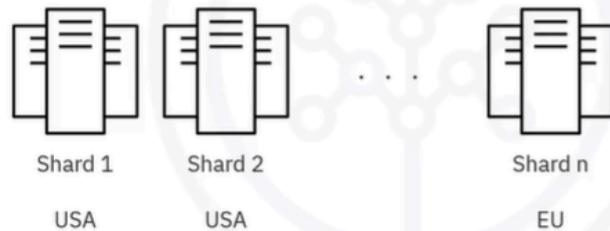
- Scale vertically (bigger, faster hardware)
- Scale horizontally (partition the data)



- After using MongoDB for some time, you will probably outgrow your hardware. For example, if you need to store even more data or you want to improve your read and write performance, naturally, you will invest in bigger and faster hardware to increase capacity, but sometimes that is not feasible. In those situations, you can scale horizontally by implementing sharding, which is the partitioning of your biggest collections.

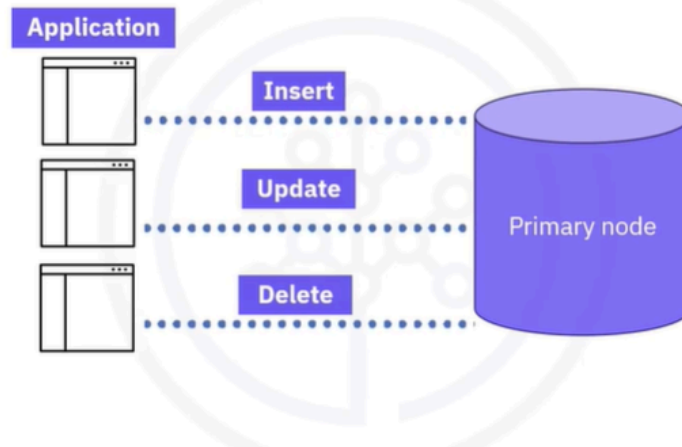
Benefits of Sharding

- Helps with increased throughput and capacity
- Can help adhere to legal requirements for data storage



- Sharding your biggest or most demanding collections has many benefits. When partitioning your data across shards, you increase your throughput by directing your queries only to relevant shards.
- You can also store more data which previously you couldn't fit on a single node. You can also split data across shards based on regions. Data for US customers will only live on shards running in the US, while European customers have their data on European base shards. As a global application, you get one view of your database to manage.

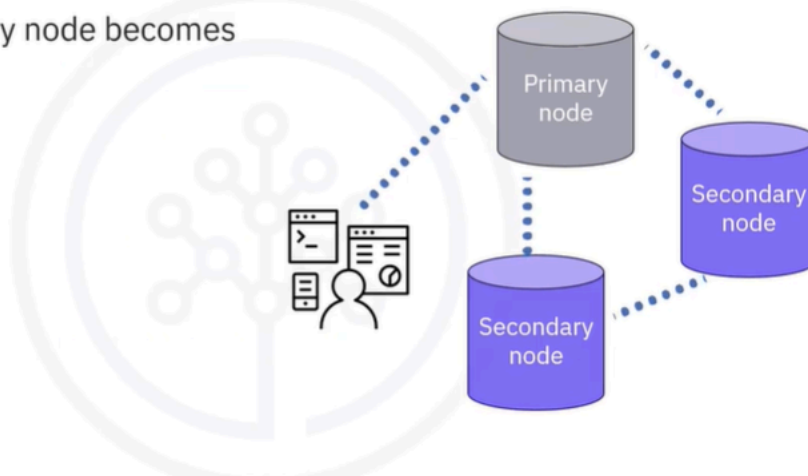
The Primary node and elections



- The primary node is the only node that can accept right operations including inserts, updates, and deletes from client applications.

Why does an election happen?

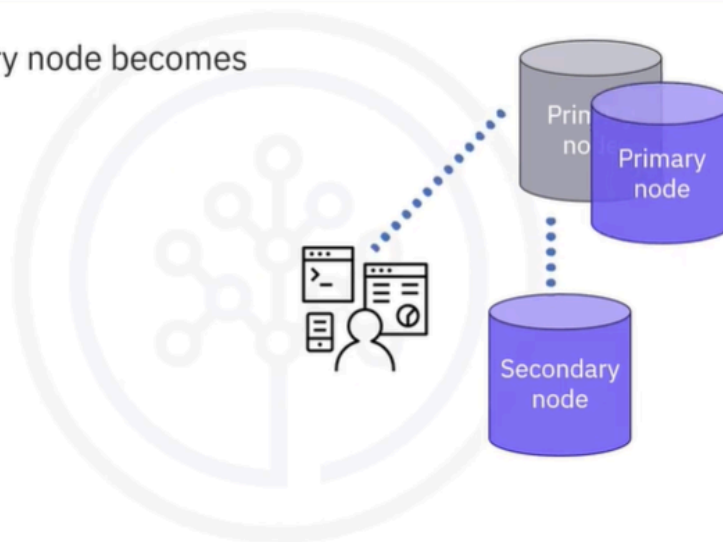
- The current primary node becomes unavailable



- An election is the process of selecting a new primary node. As MongoDB is a highly available system, MongoDB can automatically elect one of the secondary nodes as the new primary node.

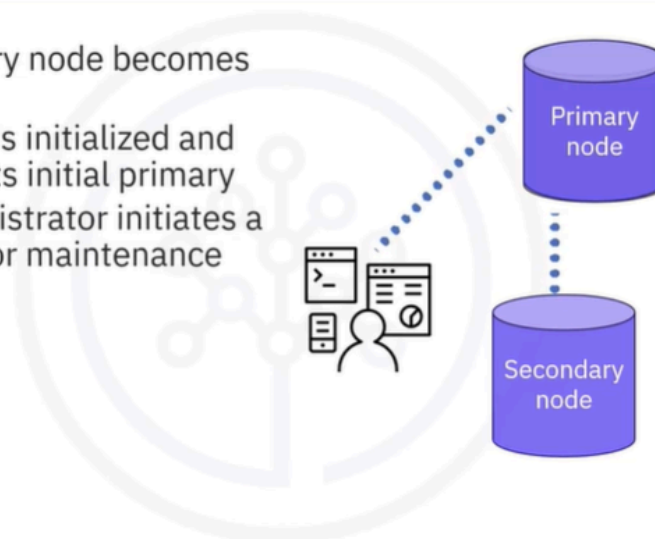
Why does an election happen?

- The current primary node becomes unavailable



Why does an election happen?

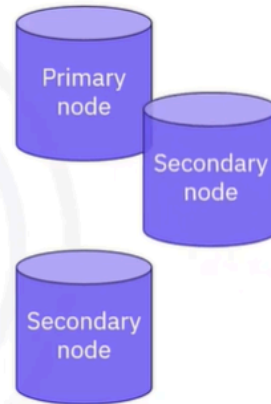
- The current primary node becomes unavailable
- A new replica set is initialized and needs to choose its initial primary
- A database administrator initiates a manual failover for maintenance or upgrades



- The election of a new primary node happens when; the current primary node becomes unavailable, a new replica set is initialized and needs to choose its initial primary node, or when a database administrator initiates a manual failover for maintenance or upgrades.

Requirements to become Primary

- ✓ MongoDB's voting system ensures only one member becomes a primary node
- ✓ Member nodes that are eligible have minimal replication lag and have the most recent data
- ✓ The member node receiving the majority of the votes becomes the new Primary.



Requirements to become Primary

- ✓ MongoDB's voting system ensures only one member becomes a primary node
- ✓ Member nodes that are eligible have minimal replication lag and have the most recent data
- ✓ The member node receiving the majority of the votes becomes the new Primary.



- MongoDB's voting system ensures only one member becomes a primary, so to speak, there is no split brain. Database replica members who are eligible and have the most recent data and minimal replication lag are eligible to become the primary node. The choice of which node becomes the primary node is based on which node member receives the most votes.

Recap

In this video, you learned that:

- Replication is the duplication of data and changes to the data
- Replication provides fault tolerance, redundancy, and high availability
- Replication does not provide disaster recovery
- You can use sharding to scale horizontally

Sure! Let's talk about replication in MongoDB.

Replication is like having multiple copies of a book in different libraries. Imagine you have a favorite book, and you want to make sure that even if one library loses it, you can still find it in another library. In MongoDB, replication works the same way. It creates copies of your data across different servers, called nodes. This means if one server goes down or has a problem, you still have access to your data from another server. This process helps keep your database available and safe, just like having backup copies of your favorite book ensures you can always read it.

How does sharding improve database performance?

Sharding improves database performance by **distributing data across multiple servers**. Here's how it works in simple terms:

- **Partitioning Data:** Sharding divides your large database into smaller, more manageable pieces called shards. Each shard holds a portion of the data.

- **Parallel Processing:** When you query the database, the system can send requests to multiple shards at the same time. This means that instead of one server handling all the requests, several servers can work together, speeding up the response time.
- **Increased Capacity:** By spreading data across different servers, sharding allows you to store more data than a single server could handle. This is especially useful when your data grows beyond the limits of one server.
- **Targeted Queries:** Sharding can also direct queries to only the relevant shards. For example, if you are looking for data related to customers in the US, the system can query only the shard that contains that data, rather than searching through all the data.

Overall, sharding helps to **scale your database** efficiently, improving both performance and capacity.

What is sharding in database management?

Sharding in database management is a method of **partitioning** a large database into smaller, more manageable pieces called **shards**. Each shard is a separate database that holds a portion of the overall data. Here are the key points about sharding:

- **Data Distribution:** Sharding distributes data across multiple servers or nodes. This helps in managing large datasets more effectively.
- **Improved Performance:** By splitting the data, sharding allows for parallel processing of queries. This means multiple servers can handle requests simultaneously, leading to faster response times.
- **Scalability:** Sharding enables horizontal scaling, meaning you can add more servers to accommodate growing data needs without overloading a single server.
- **Targeted Access:** When querying data, the system can direct requests to specific shards that contain the relevant data, reducing the amount of data that needs to be searched.

In summary, sharding is a technique used to enhance the performance, scalability, and manageability of databases by breaking them into smaller, distributed parts.